California State University, Northridge

Department of Computer Science

VLA Robot Learning

A Vision-Language-Action Model with Reinforcement Learning Feedback

PROJECT REPORT

Author Sara Aly, Connor Juman

Instructor Dr. Enno Huang

Class COMP 542 - Machine Learning

Date March 2026

Abstract

Standard Vision-Language-Action (VLA) models predict robot actions from visual observations and natural language instructions alone, ignoring the history of what the robot has already attempted. This project introduces **RLConditionedVLA**, a multimodal robot policy that conditions on three input streams simultaneously: (i) video frames encoded by a pretrained CLIP vision backbone, (ii) a task instruction encoded by the CLIP text encoder, and (iii) a structured history of *(action index, reward)* pairs from previous timesteps, encoded as dedicated sequence tokens.

These three modalities are projected to a shared 256-dimensional embedding space and fused by a six-layer causal Transformer. A learnable aggregation token placed at the end of the sequence — exploiting the causal attention mask to attend over all preceding tokens — is then passed through a two-layer classification head to predict the next discrete action.

Training proceeds in two phases: supervised fine-tuning on expert demonstrations (behavioural cloning), followed by online reinforcement learning using REINFORCE with a value baseline and a KL-divergence penalty against the cloned policy to prevent catastrophic forgetting. The architecture is designed for discrete action spaces, compatible with both simulation environments (Gymnasium, MiniGrid, FrankaKitchen) and real robot hardware via a plug-in interface.

Keywords: Vision-Language-Action model, reinforcement learning, CLIP, causal transformer, behavioural cloning, discrete action prediction, sim-to-real transfer.

Contents

Abstract 1

Project Summary 4

1 Introduction 7
- 1.1 Motivation . . . 7
- 1.2 Problem Statement . . . 7
- 1.3 Contributions . . . 7
- 1.4 Report Structure . . . 8

2 Related Work 9
- 2.1 Vision-Language Models . . . 9
- 2.2 Vision-Language-Action Models . . . 9
- 2.3 Sequence Modelling for RL . . . 9
- 2.4 RLHF and KL Regularisation . . . 9

3 System Architecture 10
- 3.1 Overview . . . 10
- 3.2 CLIP Backbone . . . 10
- 3.3 Projection to Shared Space . . . 11
- 3.4 ActionRewardHistoryEncoder . . . 11
- 3.5 Token Sequence and Causal Mask . . . 11
- 3.6 Action Classification Head . . . 12
- 3.7 Parameter Count . . . 12

4 Model Integration — Step by Step 13
- 4.1 The Core Problem: Incompatible Representations . . . 13
- 4.2 Step 1 — Encode Vision with CLIP . . . 13
- 4.3 Step 2 — Encode Language with CLIP . . . 14
- 4.4 Step 3 — Encode (Action, Reward) History . . . 14
- 4.5 Step 4 — Project to a Shared 256-dim Space . . . 15
- 4.6 Step 5 — Assemble the Token Sequence . . . 15
- 4.7 Step 6 — Fuse with a Causal Transformer . . . 16
- 4.8 Step 7 — Read Out and Classify . . . 17

4.9 Why Not a Simpler Fusion Strategy? 17
4.10 Bug Found and Fixed: CLS Token Position 18

5 Training Methodology **19**
5.1 Phase 1 — Behavioural Cloning (Supervised Fine-Tuning) 19
5.2 Phase 2 — Online RL Fine-Tuning (REINFORCE + Baseline) 20
5.3 Evaluation Protocol 20

6 Implementation Details **21**
6.1 File Structure 21
6.2 Episode Data Format 21
6.3 Environment Interface 22
6.4 Running the Pipeline 22

7 Current Project Status **23**
7.1 Experiment Log 23

8 Discussion **24**
8.1 Strengths of the Proposed Architecture 24
8.2 Limitations 24
8.3 Comparison with Related Models 25

9 Future Work & Next Steps **26**
9.1 Immediate (Phase A — Get Pipeline Running) 26
9.2 Short-term (Phase B — RL Fine-Tuning) 26
9.3 Medium-term (Phase C — Improve & Ablate) 26
9.4 Long-term (Phase D — Sim-to-Real) 26

10 Conclusion **27**

A Issues & Fixes Log **29**

B Change Log **29**

Project Summary

One-line pitch

RLConditionedVLA is a robot policy that fuses camera video, a natural-language task goal, and a history of past (action, reward) pairs through a shared causal Transformer — enabling adaptive, history-aware decision-making entirely from learned representations.

The Problem

Standard Vision-Language-Action (VLA) models — such as RT-2 and Octo — treat every decision as a *stateless* perception query: given the current image and instruction, predict the next action. This design has two fundamental limitations:

- **No recovery signal.** If the robot tries an action and fails (reward < 0), the policy has no mechanism to avoid repeating the same mistake. It cannot distinguish "first attempt" from "fifth failed attempt."
- **No RL grounding at inference.** Even when trained with RL, the standard formulation discards the reward signal at inference time — the very signal that RL training optimised for.

Our Solution

We introduce **history conditioning**: the (action index, scalar reward) pair from each of the last H timesteps is encoded as a dedicated sequence of *tokens* and fused alongside vision and language tokens in a causal Transformer. The model therefore sees *what it tried and whether it worked* before deciding what to do next.

This is implemented by **ActionRewardHistoryEncoder**, a lightweight module that:

1. Embeds each discrete action index via a learnable lookup table (categorical, not ordinal).
2. Projects each scalar reward to 256-dim via a linear layer.
3. Fuses (action + reward) into one token per timestep.
4. Adds a learnable positional embedding per history slot.

The history tokens join the vision and language tokens in a single causal sequence.

With a CLS token at the *last* position, the causal attention mask ensures it aggregates information from every other token before the action head reads it.

Training Pipeline

Training uses two sequential phases:

1. **Behavioural Cloning (Phase 1).** Cross-entropy loss on expert demonstration trajectories. Provides a stable, grounded initialisation.
2. **REINFORCE with KL Penalty (Phase 2).** Online environment interaction. A value baseline reduces gradient variance. A KL-divergence penalty against the Phase 1 checkpoint prevents catastrophic forgetting.

At a Glance

Property	Value
Vision backbone	CLIP ViT-B/32 (frozen, 86M params)
Language backbone	CLIP Text Encoder (frozen, 38M params)
History encoder	ActionRewardHistoryEncoder (trainable, ≈267K params)
Fusion module	Causal Transformer: 6 layers, 8 heads, $d = 256$
Total trainable params	≈3.8M
Action space	Discrete (N actions, configurable)
History window	$H = 4$ timesteps (configurable)
Frame buffer	$T = 3$ frames (configurable)
Phase 1 loss	Cross-entropy + label smoothing ($\epsilon = 0.05$)
Phase 2 loss	REINFORCE + value baseline + entropy + KL penalty
Environments	Any Gymnasium env with `rgb_array` render
Hardware target	Consumer GPU (real-robot stub included)

Architecture Diagram

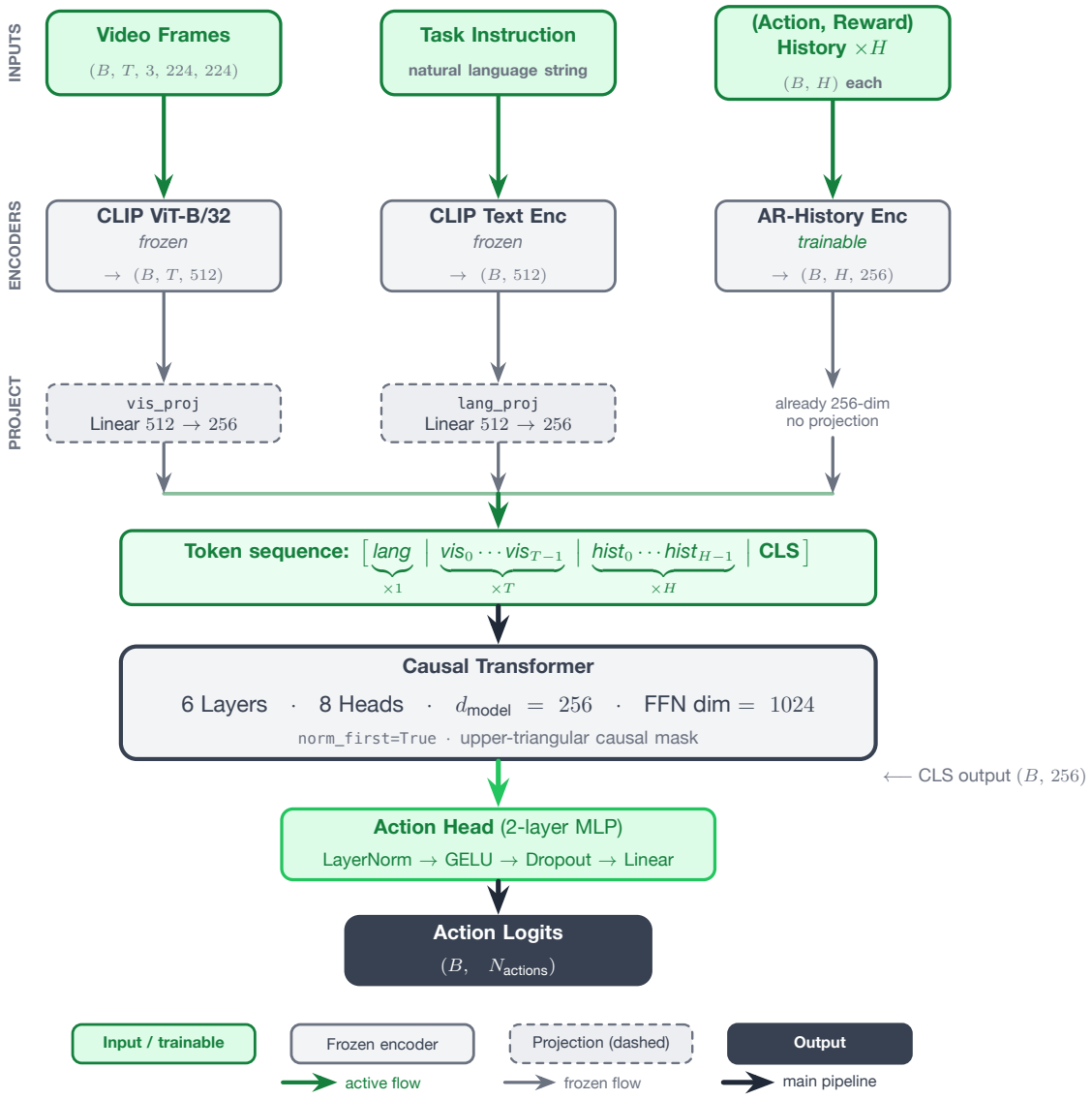

Figure 1: Full forward-pass architecture of **RLConditionedVLA**. Three input streams — video frames, a natural language instruction, and an (action, reward) history — are independently encoded, projected to a shared 256-dim space, and assembled into a causal token sequence. The **CLS** token placed last attends to all preceding tokens (vision, language, history) via the upper-triangular causal mask, and its output is decoded by a two-layer MLP into discrete action logits. Green borders = trainable or active-flow components; gray = frozen CLIP encoders; **black** = main inference pipeline. See `architecture_diagram.tex` for the standalone source.

1 Introduction

1.1 Motivation

Modern robotics increasingly relies on large pretrained models to bridge the gap between natural language instructions and physical actions. Vision-Language-Action (VLA) models such as RT-2 [1] and Octo [2] have demonstrated that a model trained on internet-scale image-text data can be fine-tuned to generate robot control signals. However, these models treat each decision as a *stateless* perception problem: given the current image and instruction, predict the action.

This design ignores a fundamentally useful signal: **what the robot just tried, and whether it worked**. If a robot attempts to grasp an object and fails (reward < 0), a history-conditioned policy can adapt its next action accordingly. Without this signal, the policy has no mechanism to recover from repeated mistakes.

1.2 Problem Statement

Given a task instruction ℓ (natural language) and the robot's observation at time t (video frame s_t), together with its recent interaction history $\mathcal{H}_t = \{(a_{t-H},\, r_{t-H}), \ldots, (a_{t-1},\, r_{t-1})\}$, predict the next action a_t from a finite discrete action set $\mathcal{A} = \{0, 1, \ldots, N-1\}$:

$$a_t = \pi_\theta(s_t,\ \ell,\ \mathcal{H}_t)$$

where π_θ is the parameterised policy to be learned.

1.3 Contributions

- A multimodal VLA architecture (**RLConditionedVLA**) that treats (action, reward) history as first-class input tokens, fused with vision and language via a causal Transformer.
- A two-phase training pipeline: behavioural cloning initialisation followed by online REINFORCE with a KL-divergence penalty to prevent forgetting.
- A modular codebase with a Gymnasium-compatible environment interface and a real-robot stub, supporting sim-to-real transfer.

- Identification and correction of a critical architectural bug (§4.10) in which the CLS aggregation token was isolated from all other input tokens by the causal mask.

1.4 Report Structure

Section 2 reviews related work. Section 3 describes the overall architecture. Section 4 explains step-by-step how the three models are combined. Section 5 describes the training methodology. Section 6 covers implementation details. Section 7 summarises the current project status. Section 8 discusses design choices. Section 9 outlines next steps. Section 10 concludes.

2 Related Work

2.1 Vision-Language Models

CLIP [3] introduced contrastive learning on 400M image-text pairs, producing a shared embedding space where visual and textual representations of the same concept are geometrically close. The Vision Transformer (ViT) [4] underpins the CLIP image encoder (ViT-B/32 in this work), partitioning images into 32×32 pixel patches processed by self-attention layers. We use CLIP's visual-semantic alignment as a frozen foundation, avoiding the need for large task-specific vision datasets.

2.2 Vision-Language-Action Models

RT-2 [1] extends a 55-billion parameter vision-language model to generate tokenised robot actions, achieving impressive zero-shot and few-shot generalisation on manipulation tasks. Octo [2] provides a smaller (93M parameter), open-source generalist policy designed specifically for fine-tuning on new tasks and embodiments. Both models treat inference as a one-shot perception problem without explicit RL history conditioning, which is the primary gap this work addresses.

2.3 Sequence Modelling for RL

The Decision Transformer [5] reframes offline RL as a conditional sequence modelling problem: given a sequence of (return-to-go, state, action) tokens, predict the next action. This work adapts that idea to the online, multimodal setting: the history of (action index, reward) pairs is encoded as tokens and injected into the Transformer's input sequence alongside vision and language tokens.

2.4 RLHF and KL Regularisation

Ouyang et al. [6] introduced KL-divergence penalties against a reference model to prevent reinforcement learning from destroying the knowledge acquired during supervised pretraining. We apply the same strategy: the behavioural cloning checkpoint serves as a frozen reference, and an explicit KL penalty keeps the RL policy from diverging too far.

3 System Architecture

3.1 Overview

RLConditionedVLA consists of four main components that are composed in a strict pipeline:

1. **CLIP Vision Encoder** — maps video frames to semantic embeddings.
2. **CLIP Text Encoder** — maps the language instruction to a single goal embedding.
3. **ActionRewardHistoryEncoder** — maps the discrete (action index, scalar reward) history to a sequence of learned token embeddings.
4. **Causal Fusion Transformer** — jointly reasons over all token types and produces the next discrete action via a classification head.

Listing 1: High-level data flow through RLConditionedVLA.

```
1 Video frames (B,T,3,224,224) --> CLIP ViT-B/32 --> vis_proj --> vis_tokens (B,T,256)
2 Language (B,77) --> CLIP text enc --> lang_proj --> lang_token (B,1,256)
3 (action,reward)xH --> AR-History-Enc --> hist_tokens (B,H,256)
4 |
5 [lang | vis_0..T-1 | hist_0..H-1 | CLS]
6 |
7 Causal Transformer (6 layers, 8 heads)
8 |
9 CLS output (B,256)
10 |
11 Action Head --> logits (B,N_actions)
```

3.2 CLIP Backbone

The CLIP ViT-B/32 model [3] provides both the vision and language encoders. The image encoder splits each 224×224 frame into $7 \times 7 = 49$ non-overlapping patches of size 32×32 pixels. Each patch is linearly projected to a 768-dimensional token, and 12 layers of multi-head self-attention produce a 512-dimensional global image embedding at the `[CLS]` position.

Frames are encoded independently (no temporal convolutions), allowing the Transformer fusion stage to learn temporal relationships from positional structure alone. The text encoder applies the same transformer architecture to 77 BPE tokens, producing a 512-dimensional sentence-level embedding at the `[EOS]` position.

3.3 Projection to Shared Space

Both CLIP outputs are projected from 512 to 256 dimensions via learnable linear layers (`vis_proj` and `lang_proj`). The history encoder natively produces 256-dimensional tokens. All three modalities therefore occupy the same embedding space before entering the fusion Transformer.

Note

The projection layers are trainable even when CLIP is frozen. They serve as a domain-adaptation interface, remapping CLIP's general-purpose embeddings towards the robot task distribution.

3.4 ActionRewardHistoryEncoder

For each of the H history timesteps $i \in \{t-H, \ldots, t-1\}$:

$$\mathbf{a}_i = \text{Embedding}(a_i;\ W_a) \in \mathbb{R}^{256} \tag{1}$$

$$\mathbf{r}_i = \text{Linear}(r_i;\ W_r) \in \mathbb{R}^{256} \tag{2}$$

$$\mathbf{h}_i = \text{LayerNorm}(\text{Linear}([\mathbf{a}_i \,\|\, \mathbf{r}_i]) + \mathbf{p}_i) \tag{3}$$

where $[\,\cdot \,\|\, \cdot\,]$ denotes concatenation, $W_a \in \mathbb{R}^{(N+1)\times 256}$ is the action embedding table (the extra index N is a learned "no-action" padding token for cold-start), $W_r \in \mathbb{R}^{1\times 256}$ is the reward projection, and $\mathbf{p}_i \in \mathbb{R}^{256}$ is a learned positional embedding indexed by history slot i.

Stacking H such vectors yields `hist_tokens` $\in \mathbb{R}^{B\times H\times 256}$.

3.5 Token Sequence and Causal Mask

The complete input sequence to the Transformer is:

$$\mathbf{X} = [\,\mathbf{e}_{\text{lang}} \,\|\, \mathbf{v}_{t-T} \cdots \mathbf{v}_{t-1} \,\|\, \mathbf{h}_{t-H} \cdots \mathbf{h}_{t-1} \,\|\, \mathbf{c}\,] \in \mathbb{R}^{(1+T+H+1)\times 256}$$

where $\mathbf{c}$ is a learnable CLS token placed **last**. An upper-triangular causal mask is applied, so position i attends only to positions $j \le i$. With CLS at the final position, it attends to every preceding token, making it a full aggregation over language, visual, and history information.

Listing 2: Causal attention mask for default config (T=3, H=4). Tick = attend, X = blocked.

```
            lang  v0   v1   v2   h0   h1   h2   h3   CLS
    lang  [  v     x    x    x    x    x    x    x    x  ]
    v0    [  v     v    x    x    x    x    x    x    x  ]
    v1    [  v     v    v    x    x    x    x    x    x  ]
    v2    [  v     v    v    v    x    x    x    x    x  ]
    h0    [  v     v    v    v    v    x    x    x    x  ]
    h1    [  v     v    v    v    v    v    x    x    x  ]
    h2    [  v     v    v    v    v    v    v    x    x  ]
    h3    [  v     v    v    v    v    v    v    v    x  ]
    CLS   [  v     v    v    v    v    v    v    v    v  ]  <- attends to everything
```

3.6 Action Classification Head

The CLS token output $\mathbf{c}' \in \mathbb{R}^{256}$ is passed through:

$$\hat{\mathbf{y}} = W_2\,\mathrm{GELU}(W_1\,\mathrm{LayerNorm}(\mathbf{c}'))\,, \quad W_1 \in \mathbb{R}^{128\times 256},\ W_2 \in \mathbb{R}^{N\times 128}$$

with dropout ($p = 0.1$) after the GELU activation. The output $\hat{\mathbf{y}} \in \mathbb{R}^N$ contains unnormalised action logits. At inference, the action is selected as $a_t = \arg\max_i \hat{y}_i$ (deterministic) or sampled from softmax($\hat{\mathbf{y}}$) (stochastic, used during RL rollouts).

3.7 Parameter Count

Table 1: Parameter breakdown of RLConditionedVLA (`freeze_clip=True`).

Component	**Parameters**	**Trainable**
CLIP ViT-B/32 image encoder	≈86.2M	No
CLIP text encoder	≈37.8M	No
`vis_proj` Linear(512→256)	131,328	Yes
`lang_proj` Linear(512→256)	131,328	Yes
ActionRewardHistoryEncoder	≈267K	Yes
CLS token (learnable vector)	256	Yes
Causal Transformer (6L, 8H)	≈3.2M	Yes
Action head (2-layer MLP)	≈33K	Yes
Total trainable	**≈3.8M**	
Total (incl. frozen)	**≈128M**	

4 Model Integration — Step by Step

This section traces a single forward pass through `RLConditionedVLA.forward()` with explicit tensor shapes and the reasoning behind each combination decision.

4.1 The Core Problem: Incompatible Representations

The three input streams cannot be directly combined because they live in different mathematical spaces:

Table 2: Raw input representations before encoding.

Modality	Raw format	Problem
Video frames	Pixel tensors $(B, T, 3, 224, 224)$, uint8	High-dimensional, spatial; no semantic structure
Language	Integer token IDs $(B, 77)$, int64	Symbolic, discrete; no overlap with pixel space
Action/reward history	Mixed: int indices + float scalars, (B, H) each	Heterogeneous types; need token-like representation

The solution is a four-step pipeline:

I. **Encode**: raw inputs $\rightarrow$ per-modality embeddings

II. **Project**: per-modality embeddings $\rightarrow \mathbb{R}^{256}$

III. **Assemble**: $\mathbb{R}^{256} \rightarrow$ token sequence

IV. **Fuse**: token sequence $\rightarrow$ action logits

4.2 Step 1 — Encode Vision with CLIP

Code: `RLConditionedVLA.encode_frames()` (line 117)

Frames are flattened from (B, T, C, H, W) to $(B{\cdot}T, C, H, W)$ and processed independently by the CLIP ViT-B/32 encoder. Each frame is divided into 49 patches; self-attention across the 49 patch tokens (plus one internal CLS token) produces a 512-dim global embedding. The $(B{\cdot}T, 512)$ output is reshaped to $(B, T, 512)$ and projected to $(B, T, 256)$ by `vis_proj`.

Why encode each frame independently? The fusion Transformer handles temporal relationships across frame tokens. Encoding frames independently keeps the CLIP encoder unchanged and avoids the need for temporally-aware fine-tuning of the vision backbone.

Why freeze CLIP? CLIP's 400M-pair pretraining produces visual embeddings where "red cube" and an image of a red cube are geometrically close. Unfreezing during early training with limited robot data would destroy this alignment faster than it could be re-learned.

4.3 Step 2 — Encode Language with CLIP

Code: `RLConditionedVLA.encode_language()` (line 129)

The CLIP tokeniser maps the instruction string to 77 BPE token IDs. The CLIP text encoder applies 12 transformer layers to these tokens, and the embedding at the `[EOS]` position captures the full sentence meaning as a 512-dim vector. This is projected to $(B, 1, 256)$ by `lang_proj`, becoming a *single token* in the fusion sequence.

Why CLIP and not BERT or T5? CLIP's text encoder was trained *jointly* with its image encoder. The sentence "pick up the red cube" has an embedding close to an image of a red cube being picked up — an alignment that BERT and T5 do not possess. Using CLIP for both modalities means the two encoders already "speak the same language" before fusion begins.

Why one token for the entire instruction? The instruction functions as a global goal context, not a word-level signal. A single token at the head of the sequence allows all subsequent visual and history tokens to attend leftward and incorporate the goal during their own update steps, without inflating the sequence length.

4.4 Step 3 — Encode (Action, Reward) History

Code: `ActionRewardHistoryEncoder.forward()` (line 41)

The action index and scalar reward for each of the H previous timesteps are encoded separately and then fused:

- **Action** (discrete int): passed through a learnable embedding table $W_a \in \mathbb{R}^{(N+1)\times 256}$. An embedding table is used because action indices are categorical — index 3 does not mean "three times more" than index 1.
- **Reward** (continuous float): projected by $W_r \in \mathbb{R}^{1\times 256}$, allowing the model to learn

which magnitude ranges are significant.

- **Fusion**: the two 256-dim vectors are concatenated and projected back to 256-dim, producing a single token per timestep where the reward *modifies* the meaning of the action.
- **Positional encoding**: a learned embedding $\mathbf{p}_i$ per history slot tells the Transformer which step is oldest and which is newest.

Cold-start padding: at the beginning of an episode the robot has no history. Missing slots are filled with action index N (a dedicated "no-action" embedding, not zero) and reward 0.0.

4.5 Step 4 — Project to a Shared 256-dim Space

Table 3: Dimension alignment before Transformer fusion.

Modality	Native dim	Projection	Output dim
CLIP vision	512	`vis_proj`	256
CLIP language	512	`lang_proj`	256
History encoder	256	(none)	256
CLS token	256	(parameter)	256

The 256-dim fusion space was chosen over CLIP's native 512 to halve the Transformer parameter count and memory footprint while remaining large enough for cross-modal reasoning across three modalities.

4.6 Step 5 — Assemble the Token Sequence

Code: `RLConditionedVLA.forward()` (line 161)

Listing 3: Token sequence assembly (forward pass).

```
sequence = torch.cat(
    [lang_tokens_,   # (B, 1, 256)
     vis_tokens,     # (B, T, 256)
     hist_tokens,    # (B, H, 256)
     cls],           # (B, 1, 256)  <-- LAST
    dim=1
)   # shape: (B, 1+T+H+1, 256)  =  (B, 9, 256) with defaults T=3, H=4
```

The ordering is deliberate:

Table 4: Rationale for token ordering in the sequence.

Position	Rationale
Language first	All later tokens attend leftward and see the goal context, enabling instruction-conditioned visual attention.
Vision next	Older frames before newer frames; the newest frame can attend to the full frame history.
History last (before CLS)	History tokens can attend to both the instruction and the visual context when updating their representations.
CLS at end	With a causal mask, the last position attends to all preceding tokens. See §4.10 for the critical bug this ordering fixes.

4.7 Step 6 — Fuse with a Causal Transformer

Code: `RLConditionedVLA.forward()` (line 165)

A `nn.TransformerEncoder` with 6 layers, 8 attention heads, and FFN dimension 1024 ($= 4 \times 256$, the standard 4$\times$ expansion from [7]) processes the token sequence with a causal mask.

Each layer computes:

$$\mathbf{X}^{(\ell+1)} = \text{LayerNorm}\big(\mathbf{X}^{(\ell)} + \text{MHA}\big(\mathbf{X}^{(\ell)}, \mathbf{M}\big) + \text{FFN}(\cdot)\big)$$

where $\mathbf{M}$ is the upper-triangular boolean mask and pre-LayerNorm ordering (`norm_first=True`) follows modern practice (GPT-2, LLaMA) for improved training stability.

After 6 layers, the CLS token at the last position has attended to all 9 tokens in the sequence. The model has learned task-specific attention patterns, for example: which visual tokens to focus on given a specific instruction, and how the reward history should modify the interpretation of the visual scene.

Why self-attention over the joint sequence rather than cross-attention? Cross-attention would require designating one modality as the "query" and the others as "keys/values" — an arbitrary choice. Joint self-attention lets every token attend to every other modality symmetrically, with the mask imposing only temporal ordering.

4.8 Step 7 — Read Out and Classify

Code: `RLConditionedVLA.forward()` (line 170)

Listing 4: CLS read-out and action classification.

```
cls_out = out[:, -1, :]          # (B, 256) -- last token = CLS
return self.action_head(cls_out) # (B, num_actions)
```

The two-layer action head (with LayerNorm, GELU, and Dropout) maps the 256-dim CLS representation to N-dimensional logits. A two-layer MLP is used rather than a single linear layer because the CLS token contains a mixed multimodal representation; the hidden layer and nonlinearity allow the model to disentangle modality-specific information before the final classification boundary.

4.9 Why Not a Simpler Fusion Strategy?

Table 5: Alternative fusion strategies considered and rejected.

Alternative	Reason for rejection
Concatenate + single MLP	Fixed fusion weights; cannot learn "attend to red tokens when instruction says red". All modalities mixed equally.
Late fusion (per-modality MLP, summed)	No cross-modal interaction before the decision. Cannot correlate "bad reward" with "what the scene looked like".
Cross-attention (history queries vision+lang)	Arbitrarily designates history as primary modality. Joint self-attention is strictly more flexible.
RNN over history	Implicit memory; gradients vanish; cannot inspect which past steps the model uses via attention weights.
Bidirectional transformer (no causal mask)	Loses temporal ordering inductive bias. Planned as ablation A6.

4.10 Bug Found and Fixed: CLS Token Position

Discovered: 2026-03-04 **Severity: Critical**

Original code (incorrect):

```
sequence = torch.cat([cls, lang_tokens_, vis_tokens, hist_tokens], dim
    =1)
cls_out  = out[:, 0, :]   # reading from position 0
```

With CLS at position 0 and a causal mask, position i can only attend to positions $j \leq i$. CLS at position 0 can only ever attend to itself — across *all* 6 Transformer layers. After L layers: $\mathbf{c}^{(L)} = f^L(\mathbf{c}^{(0)})$, a function only of the initial CLS parameter. The action head therefore received zero information from vision, language, or history. The entire CLIP encoding was ignored.

Fix (correct):

```
sequence = torch.cat([lang_tokens_, vis_tokens, hist_tokens, cls], dim
    =1)
cls_out  = out[:, -1, :]  # reading from the last position
```

At the last position, the causal mask allows CLS to attend to all preceding tokens across all layers, which is the intended aggregation behaviour.

Files fixed: `models/vla_model.py` (lines 157, 166) and `training/rl_trainer.py` (lines 217, 223).

5 Training Methodology

Training proceeds in two sequential phases. Phase 1 provides a stable, demonstration-grounded initialisation. Phase 2 refines the policy through environment interaction.

5.1 Phase 1 — Behavioural Cloning (Supervised Fine-Tuning)

Given a dataset $\mathcal{D} = \{(\mathbf{s}_t, \ell, \mathcal{H}_t, a_t^*)\}$ of expert transitions, the model is trained to imitate the expert action a_t^* via cross-entropy loss with label smoothing ϵ:

$$\mathcal{L}_{\text{BC}} = -\sum_{a=0}^{N-1} \tilde{y}_a \log \hat{y}_a, \quad \tilde{y}_a = (1-\epsilon)\,\mathbf{1}[a = a_t^*] + \frac{\epsilon}{N}$$

Label smoothing ($\epsilon = 0.05$) regularises against overconfident predictions, which is important when expert demonstrations contain ambiguous or sub-optimal actions.

Table 6: Behavioural cloning hyperparameters.

Hyperparameter	Value	Rationale
Loss	Cross-entropy	Standard for discrete classification
Label smoothing ϵ	0.05	Prevents overconfidence on noisy demos
Optimiser	AdamW	Decoupled weight decay
Learning rate	3×10^{-4}	Standard for Transformer fine-tuning
LR schedule	Cosine $\rightarrow 10^{-6}$	Smooth decay avoids late-stage oscillation
Gradient clip	1.0	Stabilises early training
Batch size	32	Fit in memory; stable gradient estimates
Epochs	50	Adjust based on dataset size
Validation split	10%	Early stopping criterion

5.2 Phase 2 — Online RL Fine-Tuning (REINFORCE + Baseline)

Starting from the Phase 1 checkpoint, the policy is refined through online interaction using the REINFORCE algorithm [8] with a learned value baseline.

Rollout collection: for each update step, $K = 4$ episodes are collected by running the stochastic policy ($\pi(a|s) = \text{softmax}(\hat{\mathbf{y}})$) in the environment.

Return computation: discounted returns are computed at each step:

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}, \quad \gamma = 0.99$$

Returns are normalised (zero mean, unit variance) within each batch to reduce gradient variance.

Composite loss:

$$\mathcal{L}_{\text{RL}} = \underbrace{-\mathbb{E}[\log \pi(a_t|s_t) \cdot (G_t - V(s_t))]}_{\text{policy loss (REINFORCE)}} + \underbrace{0.5\,\mathbb{E}\big[(V(s_t) - G_t)^2\big]}_{\text{value baseline loss}} - \underbrace{0.01\,\mathcal{H}[\pi]}_{\text{entropy bonus}} + \underbrace{0.1\,D_{\text{KL}}(\pi \,\|\, \pi_{\text{BC}})}_{\text{KL penalty}}$$

Table 7: RL loss component rationale.

Term	Coef.	Purpose
Policy loss (REINFORCE)	1.0	Increase probability of high-return actions
Value baseline loss	0.5	Reduce gradient variance via advantage
Entropy bonus	0.01	Prevent premature collapse to a single action
KL divergence penalty	0.1	Prevent catastrophic forgetting of BC policy

The KL penalty uses a frozen copy of the Phase 1 checkpoint as the reference distribution π_{BC}, following the RLHF approach of Ouyang et al. [6].

5.3 Evaluation Protocol

After each training phase, the policy is evaluated over 50 deterministic (arg max) episodes. Reported metrics: mean episode return $\pm$ std, mean episode length, and success rate (episodes where cumulative return $\geq$ threshold).

6 Implementation Details

6.1 File Structure

Listing 5: Project directory layout.

```
VLA-Robot-Learning/
  models/
    vla_model.py              -- RLConditionedVLA, ActionRewardHistoryEncoder
  training/
    sft_trainer.py            -- Phase 1: behavioural cloning
    rl_trainer.py             -- Phase 2: REINFORCE + value baseline + KL
  data/
    trajectory_dataset.py     -- TrajectoryDataset, episode I/O utilities
  envs/
    sim_env.py                -- SimEnv (Gym wrapper), RealEnv (robot stub)
  evaluation/
    evaluate.py               -- N-episode rollout and metrics
  configs/
    config.yaml               -- all hyperparameters
  REPORT.tex                  -- this document
  REPORT.md                   -- living notes (compiled to PDF via generate_report.py)
  generate_report.py          -- pandoc/xelatex PDF builder + watch mode
  references.bib              -- BibTeX entries
```

6.2 Episode Data Format

Each episode is stored as a Python dictionary:

Listing 6: Episode dictionary format.

```
episode = {
    "frames":      np.ndarray,  # (T_ep, H, W, 3) uint8
    "instruction": str,          # natural-language task goal
    "actions":     np.ndarray,  # (T_ep,) int64 -- discrete indices
    "rewards":     np.ndarray,  # (T_ep,) float32
}
```

Episodes are stored as a `.pkl` file (list of dicts). If no real data is available, `make_random_episodes()` generates synthetic episodes for end-to-end pipeline testing.

6.3 Environment Interface

All environments implement the `BaseEnv` interface:

Listing 7: BaseEnv interface.

```
class BaseEnv:
    def reset(self) -> dict:          # {"frame": ndarray, "instruction": str}
        ...
    def step(self, action: int):      # -> (obs, reward, done, info)
        ...
```

The `SimEnv` class wraps any Gymnasium environment with `render_mode="rgb_array"`. The `RealEnv` stub is ready to be wired to a physical robot SDK (camera capture and joint/gripper control).

6.4 Running the Pipeline

Listing 8: Training and evaluation commands.

```
# Phase 1: Behavioural Cloning
python -m training.sft_trainer --config configs/config.yaml

# Phase 2: RL Fine-Tuning (requires best_sft.pt from Phase 1)
python -m training.rl_trainer --config configs/config.yaml

# Evaluation (deterministic)
python -m evaluation.evaluate \
    --config configs/config.yaml \
    --checkpoint checkpoints/rl/best_rl.pt \
    --episodes 50

# Regenerate this PDF
python generate_report.py --tex
```

7 Current Project Status

Table 8: Component completion status.

Component	Status	File
Core model architecture	Done	`models/vla_model.py`
History encoder	Done	`models/vla_model.py`
CLS bug fix	Done	both model files
Trajectory dataset	Done	`data/trajectory_dataset.py`
Behavioural cloning trainer	Done	`training/sft_trainer.py`
RL fine-tuning trainer	Done	`training/rl_trainer.py`
Value head	Done	`training/rl_trainer.py`
KL penalty	Done	`training/rl_trainer.py`
Gym environment wrapper	Done	`envs/sim_env.py`
Real robot interface stub	Done	`envs/sim_env.py`
Evaluation script	Done	`evaluation/evaluate.py`
Config file	Done	`configs/config.yaml`
Real demonstration data	Pending	—
Gym environment selection	Pending	—
SFT training run	Pending	—
RL training run	Pending	—
TensorBoard logging	Pending	—
PPO upgrade	Pending	—

7.1 Experiment Log

Table 9: Experiment results (to be filled in as experiments are run).

Date	Phase	Environment	Epochs	Val Acc / Return	Notes
—	—	—	—	—	Pending

8 Discussion

8.1 Strengths of the Proposed Architecture

Explicit RL memory. By encoding (action, reward) history as explicit tokens, the model's "reasoning" over past experience is inspectable: attention weights between the CLS token and specific history tokens reveal which previous steps the model considers most relevant to the current decision. This is impossible with RNN-based memory.

CLIP alignment. Using CLIP for both vision and language encoders means the embeddings arrive in a pre-aligned space. The fusion Transformer does not need to learn to bridge the visual-linguistic modality gap from scratch — it only needs to learn task-specific routing.

Modular design. The three encoders are decoupled. Replacing the CLIP vision backbone with a domain-specific encoder (e.g., a depth-image encoder for indoor navigation) requires only changing `encode_frames()` and `vis_proj`.

8.2 Limitations

Global visual token only. CLIP's `[CLS]` output is a *global* image embedding — it has no spatial resolution. Tasks requiring precise spatial reasoning (e.g., grasping a specific part of an object) may benefit from using CLIP's per-patch tokens instead.

Discrete actions only (current). Continuous control (joint angles, end-effector velocities) is not currently supported. Adding a regression head or tokenising continuous actions in the RT-2 style is a planned extension.

REINFORCE variance. REINFORCE has high gradient variance even with a baseline. PPO [9] provides a clipped surrogate objective that is more stable for long episodes and sparse reward settings.

8.3 Comparison with Related Models

Table 10: Comparison with related VLA models.

Model	Params	Action type	RL history	Open source	GPU req.
RT-2	55B	Tokenised cont.	No	No	TPU
Octo	93M	Continuous	No	Yes	A100
Ours	**3.8M trainable**	**Discrete**	**Yes**	**Yes**	**Consumer GPU**

9 Future Work & Next Steps

9.1 Immediate (Phase A — Get Pipeline Running)

1. **Choose simulation environment.** Candidates: MiniGrid-Empty-5x5 (simplest discrete action space), FetchReach-v2 (realistic manipulation), FrankaKitchen-v1 (hardest).
2. **Define action space.** Enumerate the exact N discrete actions; update `config.yaml`.
3. **Collect demonstration data.** Run a scripted or random policy in the chosen environment; save as `data/episodes.pkl`.
4. **Run Phase 1 SFT** and verify validation accuracy is non-trivial.
5. **Evaluate Phase 1 checkpoint** on 50 episodes; record results in Table 8.

9.2 Short-term (Phase B — RL Fine-Tuning)

1. Run Phase 2 RL; monitor mean return per epoch.
2. Add TensorBoard logging to both trainers.
3. Compare Phase 1 and Phase 2 checkpoints on the same evaluation set.

9.3 Medium-term (Phase C — Improve & Ablate)

9.4 Long-term (Phase D — Sim-to-Real)

1. Enable domain randomisation (`env.domain_randomization: true`); tune observation noise.
2. Implement `RealEnv` with physical camera and robot SDK.
3. Evaluate on real robot; report success rate.

Table 11: Planned ablation studies.

#	Ablation	Change	Expected outcome
A1	No history	`history_len=0`	Lower recovery performance
A2	Action-only hist.	Remove reward from encoder	Slightly worse
A3	Unfreeze CLIP	`freeze_clip: false`	Better with large data
A4	PPO vs REINFORCE	Replace `rl_trainer` algorithm	More stable training
A5	History length	$H \in \{1, 2, 4, 8\}$	Find optimal window
A6	Bidirectional attn.	Remove causal mask	Marginal change

10 Conclusion

This report documents the design, implementation, and current status of **RLConditionedVLA**, a Vision-Language-Action model that conditions robot action prediction on an explicit history of (action, reward) pairs.

The key architectural contribution is the treatment of RL feedback as first-class input tokens, fused with CLIP-encoded vision and language features via a six-layer causal Transformer. This explicit, token-based memory is inspectable (via attention weights), cold-start-robust (via dedicated padding tokens), and naturally extensible to longer histories or multi-step action generation.

A critical bug — in which the CLS aggregation token was isolated from all other tokens by the causal mask — was identified and corrected during architectural analysis, underscoring the importance of carefully reasoning about the interaction between positional ordering and attention masking.

The full training pipeline (behavioural cloning followed by REINFORCE with KL regularisation) and evaluation framework are implemented and ready for execution once demonstration data and a target environment are selected.

Next action item: select a Gymnasium environment and collect the first batch of demonstration episodes (Phase A, Step 1).

References

[1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning (CoRL)*, 2023.

[2] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. You, C. Finn, and S. Levine, "Octo: An open-source generalist robot policy," in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.

[3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pp. 8748–8763, PMLR, 2021.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[5] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 15084–15097, 2021.

[6] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 27730–27744, 2022.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[8] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," vol. 8, pp. 229–256, Springer, 1992.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

A Issues & Fixes Log

Table 12: All bugs and fixes recorded chronologically.

Date	Issue	Fix	File
2026-03-03	`--highlight-style` deprecated in pandoc 3.9; LaTeX error from complex YAML `header-includes`	Removed `header-includes`; passed all styling as pandoc `-V` CLI variables	`generate_report.py`
2026-03-04	CLS token at position 0 with causal mask is isolated — never attends to vision, language, or history tokens across any layer	Moved CLS to last position; read `out[:, -1, :]`	`models/vla_model.py`, `training/rl_trainer.py`

B Change Log

Table 13: Project change log.

Date	Change
2026-03-03	Project initialised. Full codebase scaffolded: model, SFT trainer, RL trainer, dataset, sim env, evaluation, config, PDF report generator.
2026-03-04	`REPORT.md` restructured with detailed architecture diagrams, token sequence layout, parameter count table, and prioritised next steps.
2026-03-04	`REPORT.tex` created: full academic-format LaTeX report with title page, abstract, TOC, and all sections.
2026-03-04	Bug fixed: CLS token at position 0 isolated by causal mask. Moved to last position in `models/vla_model.py` and `training/rl_trainer.py`.